Verifying Rust code by translation to functional code

Jacques-Henri Jourdan

March 4th, 2026

A functional translation
Rust as a functional language
Mutable borrows
Causality loops
Proving soundness of the translation

Agenda

- Next week 2026/03/11: **exam**
- Yesterday 2026/03/03: deadline for programming project

Introduction

Program verification is hard.

A functional translation
Rust as a functional language
Mutable borrows
Causality loops
Proving soundness of the translation

Introduction

Program verification is hard.

But some tasks can be made easier by using the right tools.

The choice of a good programming language is important.

A functional translation
Rust as a functional language
Mutable borrows
Causality loops
Proving soundness of the translation

Verification of low-level software

Verification of low-level software is particularly difficult.

This is because they need control:

- over memory layout;
- over when the memory is allocated/deallocated;
- over what computation is done and when...

and many good verification languages/tools don't offer this control!

A functional translation
Rust as a functional language
Mutable borrows
Causality loops
Proving soundness of the translation

The problem of aliasing

Typically, low-level software uses pointers.

Programs with pointers are difficult to verify, because of aliasing.

Example:

```
void array_copy(int* src, int* dst, int len) {
  for(int i = 0; i < len; i++) {
    dst[i] = src[i]
  }
}
```

What spec?

The problem of aliasing

Typically, low-level software uses pointers.

Programs with pointers are difficult to verify, because of aliasing.

Example:

```
// Ensures: dst[0..len] = old(src[0..len])
// Ensures: src[0..len] = old(src[0..len])
void array_copy(int* src, int* dst, int len) {
  for(int i = 0; i < len; i++) {
    dst[i] = src[i]
  }
}
```

Wrong!

The problem of aliasing

Typically, low-level software uses pointers.

Programs with pointers are difficult to verify, because of aliasing.

Example:

```
// Requires: distinct_memory(dst[0..len], src[0..len])
// Ensures: dst[0..len] = old(src[0..len])
// Ensures: src[0..len] = old(src[0..len])
void array_copy(int* src, int* dst, int len) {
  for(int i = 0; i < len; i++) {
    dst[i] = src[i]
  }
}
```

We need to add non-aliasing hypotheses between all pairs of pointers.

- Trivial, but tedious to write and prove.
- Badly handled by automatic provers.

Our claim

Rust is well-suited for verification of low-level software.

It has a rich type system.

- Safety guarantees.
  - No need to prove safety.
- Non-aliasing guarantees.
  - Helps to reason with pointers.
- Type traits (i.e., type classes): abstraction and genericity mechanism.
  - Easy to write reusable specifications.

In this course, we show one of the techniques that can be used to verify Rust code.

- Used in a verification tool: Creusot.
- Soundness is subtle, we will study it.

1 A functional translation
- Rust as a functional language
- Mutable borrows

2 Causality loops

3 Proving soundness of the translation

Our big secret

Rust is a functional* programming language

... and this greatly helps verification.

*some squinting required.

Rust as a functional language

Local variables

```
fn incr(mut x: u64, y: u64) -> (u64, u64) {
  x += y;
  do_stuff();
  (x, y)
}
```

```
let incr x y =
  let x = x + y in
  do_stuff ();
  (x, y)
```

Rust as a functional language

Local variables

```
fn incr(mut x: u64, y: u64) -> (u64, u64) {
  x += y;
  do_stuff();
  (x, y)
}
```

```
let incr x y =
  let x = x + y in
  do_stuff ();
  (x, y)
```

As long as there is no pointer, mutating variables = shadowing.

We give a name to the value of each variable at each program point.
Variables can only be mutated directly, we can track their modifications.

Rust as a functional language

Owned pointers

`Box<T>` is Rust's type for owned pointers.

- I.e., pointers that are allocated on the heap ($\simeq$ `malloc`).
- Type system guarantee: no alias.

```
fn incr(x: Box<u64>, y: Box<u64>)
      -> (Box<u64>, Box<u64>) {
  *x += *y;
  do_stuff();
  (x, y)
}
```

```
let incr x y =
  let x = x + y in
  do_stuff();
  (x, y)
```

Because there is no alias, we know nobody else can mutate `x` and `y`.

Owned pointers functionally behave like their content.

Rust as a functional language

Shared borrows

`&T` is Rust's type for temporarily immutable pointers.

```
fn f() -> i32 {
  let x = 12;
  let a : &i32 = &x;
  let b : &i32 = a;  // allowed alias
  do_stuff(a, b);
  let y = *a + *b;
  y
}
```

```
let f () =
  let x = 12 in
  let a = x in
  let b = a in
  do_stuff(a, b);
  let y = a + b in
  y
```

Because of immutability, shared borrows `&T` functionally behave like their contents.

Summarizing

Rust aliasing rule

The reason all this works is that Rust enforces an invariant:

Mutation XOR aliasing

i.e., a memory location that is pointed to by several pointers cannot be mutated.

Consequence: there is no "action at a distance", and it is easy to track changes of variables.

Summarizing

Rust aliasing rule

The reason all this works is that Rust enforces an invariant:

Mutation XOR aliasing

i.e., a memory location that is pointed to by several pointers cannot be mutated.

Consequence: there is no "action at a distance", and it is easy to track changes of variables.

We can see such a program as a functional program.

- Creusot: translates Rust programs into a functional program.

Mutable borrows

Rust also has a type `&mut T` of mutable borrows.
Gives temporary mutable access to a variable, without aliasing.

- (Disables direct access to the variable, so the aliasing rule is still followed.)

Mutable borrows

Rust also has a type `&mut T` of mutable borrows.
Gives temporary mutable access to a variable, without aliasing.

- (Disables direct access to the variable, so the aliasing rule is still followed.)

Examples:

```
// Vec<T>: extensible array.

// Add a new element
fn push<T>(v: &mut Vec<T>, value: T) { ... }

// Get a mutable reference to the element at index idx
// e.g., to mutate it.
fn index_mut<T>(v: &mut Vec<T>, idx: usize) -> &mut T { ... }
```

Mutable borrows

Functional translation

```
fn push<T>(v: &mut Vec<T>, value: T) { ... }
fn index_mut<T>(v: &mut Vec<T>, idx: usize) -> &mut T { ... }
```

How do we translate these uses of these functions?

```
fn f() {
   let v = vec![1, 2];

   push(&mut v, 2);

   *index_mut(&mut v, 2) += 1;
   // I.e., v[2] += 1;
}
```

```
let f =
  let v = [1, 2];

  ??? push(v, 2); ???

  ??? index_mut(v, 2) + 1 ???
```

Prophecies

Our trick to solve this problem is to use a prophecy.

- a "variable" that we can read *before* assigning.
  i.e., its value depends on the future of the execution.

A typical proof that uses a prophecy variable:

1. Creation of the prophecy early in the program's execution
   - A value is fixed. We don't have any information on it.
2. Some proof steps which depend on the value of the prophecy.
3. Resolution
   - We *assign* a value to the prophecy.
   - In the proof, we learn that its (past) value is equal to that value.

Prophecies and mutable borrows

`x: &mut T` is translated into a pair:

- of the current value noted `*x`,
- and the prophetized final value of the borrow, noted `^x`.

The prophecy is used to assign its new value to the borrowed variable.

Prophecies and mutable borrows

Example: `push`

```
#[ensures(^v = *v ++ [value])]
fn push<T>(v: &mut Vec<T>, value: T) { ... }
```

Prophecies and mutable borrows

Example: push

```
#[ensures(^v = *v ++ [value])]
fn push<T>(v: &mut Vec<T>, value: T) { ... }
```

```
fn f() -> Vec<i32> {
   let v = vec![1, 2];

   let v_bor = &mut v;


   push(v_bor, 2);



   v
}
```

```
let f =
  let v = [1, 2];

  let v_proph = any in
  let v_bor = (v, v_proph) in


  push(v_bor, 2);


  let v = v_proph in


  v
```

Verifying Rust code by translation to functional code
Jacques-Henri Jourdan
A functional translation
Rust as a functional language
Mutable borrows
Causality loops
Proving soundness of the translation

Prophecies and mutable borrows

Example: push

```
#[ensures(^v = *v ++ [value])]
fn push<T>(v: &mut Vec<T>, value: T) { ... }
```

```
fn f() -> Vec<i32> {
   let v = vec![1, 2];

   let v_bor = &mut v;


   push(v_bor, 2);



   v
}
```

```
let f =
  let v = [1, 2];

  let v_proph = any in
  let v_bor = (v, v_proph) in


  push(v_bor, 2);
  (* We now know that "snd v_bor == v ++ [2]" *)

  let v = v_proph in

  (* So we can deduce that "v == [1, 2, 2]", as expected! *)
  v
```

Prophecies and mutable borrows

Example: push

```
#[ensures(^v = *v ++ [value])]
fn push<T>(v: &mut Vec<T>, value: T) { ... }
```

```
fn f() -> Vec<i32> {
   let v = vec![1, 2];

   let v_bor = &mut v;


   push(v_bor, 2);




   v
}
```

```
let f =
  let v = [1, 2];

  let v_proph = any in
  let v_bor = (v, v_proph) in
  let v = v_proph in

  push(v_bor, 2);




  v
```

Prophecies and mutable borrows

Borrowing a variable

To conclude, borrowing a variable is translated by

- Creating a new prophecy for the borrow (`let v_proph = any in`)
- Use the current value of the borrowed variable and the prophecy to create the new borrow (`let v_bor = (v, v_proph) in`).
- Use the prophecy as the new value of the borrowed variable (`let v = v_proph in`).

Prophecies and mutable borrows

Example: `index_mut`

```
#[ensures((*v)[idx] = *result)]
#[ensures((^v)[idx] = ^result)]
#[ensures(forall<i: Int> i != idx ==> (*v)[i] == (^v)[i])]
fn index_mut<T>(v: &mut Vec<T>, idx: usize) -> &mut T { ... }
```

Prophecies and mutable borrows

Example: index_mut

```
#[ensures((*v)[idx] = *result)]
#[ensures((^v)[idx] = ^result)]
#[ensures(forall<i: Int> i != idx ==> (*v)[i] == (^v)[i])]
fn index_mut<T>(v: &mut Vec<T>, idx: usize) -> &mut T { ... }
```

```
fn f() -> Vec<i32> {
   let v = vec![1, 2, 2];

   let v_bor = &mut v;

   let x = index_mut(v_bor, 2);

   *x += 1;

   v
}
```

```
let f =
  let v = [1, 2, 2];

  let v_bor = (v, any) in
  let v = snd v_bor in

  let x = index_mut(v_bor, 2);

  let x = (fst x + 1, snd x) in
  (* x will no longer be used, we resolve its prophecy. *)
  assume (fst x = snd x);

  v
```

Verifying Rust code by translation to functional code

Jacques-Henri Jourdan

A functional translation

Rust as a functional language

Mutable borrows

Causality loops

Proving soundness of the translation

Prophecies and mutable borrows

Example: index_mut

```
#[ensures((*v)[idx] = *result)]
#[ensures((^v)[idx] = ^result)]
#[ensures(forall<i: Int> i != idx ==> (*v)[i] == (^v)[i])]
fn index_mut<T>(v: &mut Vec<T>, idx: usize) -> &mut T { ... }
```

```
fn f() -> Vec<i32> {
   let v = vec![1, 2, 2];

   let v_bor = &mut v;


   let x = index_mut(v_bor, 2);


   *x += 1;


   v
}
```

```
let f =
  let v = [1, 2, 2];

  let v_bor = (v, any) in
  let v = snd v_bor in

  let x = index_mut(v_bor, 2);
  (* We learn "fst x = 2" and "v = [1, 2, snd x]". *)

  let x = (fst x + 1, snd x) in

  assume (fst x = snd x);
  (* We learn "3 = fst x = snd x = v[2]" *)
  v (* == [1, 2, 3] *)
```

Prophecies and mutable borrows

Example: `max_or_incr`

```
fn max_or_incr(ma: &mut i32, mb: &mut i32)
  -> &mut i32 {
    if *ma >= *mb {
        *mb += 42;
        ma
    } else {
        *ma += 18;
        mb
    }
}
```

Could you give the post-condition to this function?

Prophecies and mutable borrows

Example: max_or_incr

```
#[ensures(if *ma >= *mb {
            result == ma && ^mb = *mb + 42
          } else {
            result == mb && ^ma = *ma + 18
          })]
fn max_or_incr(ma: &mut i32, mb: &mut i32)
  -> &mut i32 {
    if *ma >= *mb {
        *mb += 42;

        ma
    } else {
        *ma += 18;

        mb
    }
}
```

Prophecies and mutable borrows

Example: max_or_incr

```
#[ensures(if *ma >= *mb {
            result == ma && ^mb = *mb + 42
          } else {
            result == mb && ^ma = *ma + 18
          })]
fn max_or_incr(ma: &mut i32, mb: &mut i32)
  -> &mut i32 {
    if *ma >= *mb {
        *mb += 42;

        ma
    } else {
        *ma += 18;

        mb
    }
}
```

```
let max_or_incr ma mb =

  if fst ma >= fst mb then
    let mb = (fst mb + 42, snd mb) in
    assume (fst mb = snd mb); (* resolve mb *)
    ma
  else
    let ma = (fst ma + 18, snd ma) in
    assume (fst ma = snd ma); (* resolve ma *)
    mb
```

Reborrowing

```
fn f(v: &mut Vec<i32>) {
  // v = (v₀, v_∞)
  let w = &mut *v; // Reborrowing

  w.push(1);

  // Expected: v_∞ = v₀ ++ [1]
}
```

```
let f v =

  let w = ??? in
  ???
  push(w, 1);

  assume(fst v = snd v)
```

Reborrowing

```
fn f(v: &mut Vec<i32>) {
  // v = (v_0, v_∞)
  let w = &mut *v; // Reborrowing

  w.push(1);

  // Expected: v_∞ = v_0 ++ [1]
}
```

```
let f v =

  let w = ??? in
  ???
  push(w, 1); (* Postcond.: snd w = fst w ++ [1] *)

  assume(fst v = snd v)
```

Reborrowing

```
fn f(v: &mut Vec<i32>) {
  // v = (v_0, v_∞)
  let w = &mut *v; // Reborrowing

  w.push(1);

  // Expected: v_∞ = v_0 ++ [1]
}
```

```
let f v =

  let w = (fst v, ???) in (* Init. value of w: cur. value of v *)
  ???
  push(w, 1); (* Postcond.: snd w = v_0 ++ [1]
                 I.e., we expect snd w = v_∞ *)
  assume(fst v = snd v)
```

Reborrowing

```
fn f(v: &mut Vec<i32>) {
  // v = (v_0, v_∞)
  let w = &mut *v; // Reborrowing

  w.push(1);

  // Expected: v_∞ = v_0 ++ [1]
}
```

```
let f v =

  let w = (fst v, any) in (* Final value of w: new prophecy *)
  let v = (snd w, snd v) in
  push(w, 1); (* Postcond.: snd w = v_0 ++ [1]
                 I.e., fst v = v_0 ++ [1] *)
  assume(fst v = snd v) (* I.e., v_∞ = v_0 ++ [1] *)
```

Reborrowing

To conclude, a reborrow is translated by:

- Creating a new borrow with a fresh prophecy and the current value of the initial borrow (`let w = (fst v, any) in`).
- Assigning the initial borrow with the prophecy.
  - Intuition: when the short borrow ends, the value of the long borrow is equal to the final value of the short borrow

This is actually very similar to borrowing a local variable!

Reborrowing

To conclude, a reborrow is translated by:

- Creating a new borrow with a fresh prophecy and the current value of the initial borrow (`let w = (fst v, any) in`).
- Assigning the initial borrow with the prophecy.
  - Intuition: when the short borrow ends, the value of the long borrow is equal to the final value of the short borrow

This is actually very similar to borrowing a local variable!

The same idea also works for reborrowing in depth: `&mut **x`, `&mut ***x`, `&mut (*x).0`, etc...

Projection

```
#[ensures(???)]
fn project<T, U>(x: &mut (T, U)) -> &mut T {
    &mut x.0
}
```

Projection

```
#[ensures(???)]
fn project<T, U>(x: &mut (T, U))
  -> &mut T {
    let y = &mut x.0;  // Reborrowing


    // Resolve x
    y
}
```

We consider this equivalent function.

Projection

```
#[ensures(???)]
fn project<T, U>(x: &mut (T, U))
  -> &mut T {
    let y = &mut x.0;  // Reborrowing


    // Resolve x
    y
}
```

```
let project x =

  let y = (fst (fst x), any) in
  let x = ((snd y, snd (fst x)), snd x) in


  assume(fst x = snd x)
  y
```

Projection

```
#[ensures(???)]
fn project<T, U>(x: &mut (T, U))
  -> &mut T {
    let y = &mut x.0;  // Reborrowing


    // Resolve x
    y
}
```

```
let project x = (* x = ((x00, x01), (x∞0, x∞1)) *)

  let y = (fst (fst x), any) in (* y = (x00, α) *)
  let x = ((snd y, snd (fst x)), snd x) in
  (* x = ((α, x01), (x∞0, x∞1)) *)

  assume(fst x = snd x) (* I.e., α = x∞0 ∧ x01 = x∞1 *)
  y  (* = (x00, x∞0) *)
```

If the input value is $((x_{00}, x_{01}), (x_{\infty 0}, x_{\infty 1}))$,

- then the returned value is $(x_{00}, x_{\infty 0})$,
- and we learned $x_{01} = x_{\infty 1}$.

Projection

```
#[ensures(*result == (*x).0 && ^result == (^x).0)]
#[ensures((*x).1 == (^x).1)]
fn project<T, U>(x: &mut (T, U))
  -> &mut T {
    &mut x.0
}
```

I.e.:

- Both the current value and the prophecy are projected for the return value
- The second component will never be used
  - We partially resolve the second component.

Unnesting

```
#[ensures(???)]
fn unnest<T>(x: &mut &mut T) -> &mut T {
    *x
}
```

Unnesting

```
#[ensures(???)]
fn unnest<T>(x: &mut &mut T) -> &mut T {
    let y = &mut **x;  // Reborrowing


    // Resolve x
    y
}
```

Unnesting

```
#[ensures(???)]
fn unnest<T>(x: &mut &mut T) -> &mut T {
    let y = &mut **x;  // Reborrowing


    // Resolve x
    y
}
```

```
let unnest x =
  let y = (fst (fst x), any) in
  let x = ((snd y, snd (fst x)), snd x) in

  assume(fst x = snd x)
  y
```

Unnesting

```
#[ensures(*result == **x && ^result == *^x)]
#[ensures(^*x == ^^x)]
fn unnest<T>(x: &mut &mut T) -> &mut T {
    *x
}
```

I.e.:

- We return the borrow created from the current values
- We partially resolve the prophecy part
  - Intuition: the inner borrow will never change, so the current value of the prophecy and its final value are the same.

1 A functional translation
- Rust as a functional language
- Mutable borrows

2 Causality loops

3 Proving soundness of the translation

Back to the future

So, prophecy variables let us look in the future...

Back to the future

So, prophecy variables let us look in the future...

Have you already seen science fiction movies where you can travel in time?
Is this consistent?

Back to the future

So, prophecy variables let us look in the future...

Have you already seen science fiction movies where you can travel in time?
Is this consistent?

Well, it depends on the movie.
The basic idea is that if we can modify a borrow in a way which is contradictory with its prophecy, then we can prove false.
This is a causality loop.

Back to the future

So, prophecy variables let us look in the future...

Have you already seen science fiction movies where you can travel in time?
Is this consistent?

Well, it depends on the movie.
The basic idea is that if we can modify a borrow in a way which is contradictory with its prophecy, then we can prove false.
This is a causality loop.

In the case of our translation, do we have causality loops?

Snapshots

In Creusot, the type `Snapshot<T>` is a ghost type.
At runtime, it does not contain anything (zero-size type).
But for the proof, it contains a logical value of type `T`.

Example:

```
fn f(mut x: i32) {
  let x_snap = snapshot!( x /* Can write any logical term. */ );
  x += 1;
  proof_assert!(x == *x_snap + 1);
}
```

Snapshots

In Creusot, the type `Snapshot<T>` is a ghost type.
At runtime, it does not contain anything (zero-size type).
But for the proof, it contains a logical value of type `T`.

Example:

```
fn f(mut x: i32) {
  let x_snap = snapshot!( x /* Can write any logical term. */ );
  x += 1;
  proof_assert!(x == *x_snap + 1);
}
```

Very useful to write precise specifications!

Causality loop with snapshots

```
let mut x: Snapshot<bool> = snapshot!( true );  // x = true
let bor: &mut Snapshot<bool> = &mut x;          // x = α    bor = (true, α)
*bor = snapshot!( ! ^bor );                     // x = α    bor = (!α, α)
// Resolve bor                                  // !α = α
                                                // CONTRADICTION!
```

Causality loop with snapshots

```
let mut x: Snapshot<bool> = snapshot!( true );  // x = true
let bor: &mut Snapshot<bool> = &mut x;          // x = α    bor = (true, α)
*bor = snapshot!( ! ^bor );                     // x = α    bor = (!α, α)
// Resolve bor                                  // !α = α
                                                // CONTRADICTION!
```

Conclusions:

- The soundness of this functional translation with prophecies is subtle and needs to be proved.
- "Naive" snapshots require special care.

1 A functional translation
- Rust as a functional language
- Mutable borrows

2 Causality loops

3 Proving soundness of the translation

Proving soundness of the translation

The translation heavily relies on non-aliasing guarantees.

So clearly, our proof relies on a soundness proof of the type system.

Proving soundness of the translation

The translation heavily relies on non-aliasing guarantees.

So clearly, our proof relies on a soundness proof of the type system.

In fact, our proof generalizes the proof we have sketched during the previous course.

Recap from last course

Recall: for proving the soundness of the type system, we presented syntactic typing judgements:

$$\mathsf{E}; \mathsf{L} \mid \mathsf{T}_1 \vdash \mathit{I} \dashv x. \mathsf{T}_2$$

and we gave them semantic counterparts:

$$\mathsf{E}; \mathsf{L} \mid \mathsf{T}_1 \models \mathit{I} \Rightarrow x. \mathsf{T}_2$$

Recap from last course

Recall: for proving the soundness of the type system, we presented syntactic typing judgements:

$$\mathsf{E}; \mathsf{L} \mid \mathsf{T}_1 \vdash \mathit{I} \dashv x.\, \mathsf{T}_2$$

and we gave them semantic counterparts:

$$\mathsf{E}; \mathsf{L} \mid \mathsf{T}_1 \models \mathit{I} \Rightarrow x.\, \mathsf{T}_2$$

We then proved:

- The fundamental theorem of the logical relation: $\ldots \vdash \ldots \Longrightarrow \ldots \models \ldots$
- Adequacy: $\ldots \models \ldots \Longrightarrow$ no crash

Proving soundness of the translation

We introduce a "type-spec" judgements:

$$\mathsf{E}; \mathsf{L} \mid \mathsf{T}_1 \vdash l \dashv x.\, \mathsf{T}_2 \rightsquigarrow \Phi$$

where Φ is a predicate transformer: $\Phi : (\lfloor \mathsf{T}_2 \rfloor \to \mathbf{Prop}) \to (\lfloor \mathsf{T}_1 \rfloor \to \mathbf{Prop})$

Φ translates a post-condition of l into a corresponding pre-condition.

Proving soundness of the translation

We introduce a "type-spec" judgements:

$$\mathsf{E}; \mathsf{L} \mid \mathsf{T}_1 \vdash I \dashv x.\, \mathsf{T}_2 \rightsquigarrow \Phi$$

where Φ is a predicate transformer: $\Phi : (|\mathsf{T}_2| \to \mathbf{Prop}) \to (|\mathsf{T}_1| \to \mathbf{Prop})$

Φ translates a p

I presented a translation to a functional language, but now I am presenting judgements for predicate transformers, which generate logical specifications.

Why are my type-spec judgements still relevant?

Proving soundness of the translation

We introduce a "type-spec" judgements:

where Φ is a pr

Φ translates a p

I presented a translation to a functional language, but now I am presenting judgements for predicate transformers, which generate logical specifications.

Through the use of a weakest-precondition calculus, it is easy to go from a functional program to a specification.

And, anyway, we are doing all this only for verification, so the functional program is only useful for its WP.

Thus, in the proof, we only consider predicate transformers, and we check that they match the WP of the functional program we generate.

Why, in the proof, do we prefer predicate transformers?

Proving soundness of the translation

We introduce a

where Φ is a pr

Φ translates a p

I presented a translation to a functional language, but now I am presenting judgements for predicate transformers, which generate logical specifications.

Through the use of a weakest-precondition calculus, it is easy to go from a functional program to a specification.

And, anyway, we are doing all this only for verification, so the functional program is only useful for its WP.

Thus, in the proof, we only consider predicate transformers, and we check that they match the WP of the functional program we generate.

In the proof, we use predicate transformers because then we do not have to deal with details of the operational semantics of the functional language (steps, ...).

Proving soundness of the translation

We introduce a "type-spec" judgements:

$$\mathsf{E}; \mathsf{L} \mid \mathsf{T}_1 \vdash \mathit{I} \dashv x.\, \mathsf{T}_2 \rightsquigarrow \Phi$$

where Φ is a predicate transformer: $\Phi : (\lfloor \mathsf{T}_2 \rfloor \to \mathbf{Prop}) \to (\lfloor \mathsf{T}_1 \rfloor \to \mathbf{Prop})$

Φ translates a post-condition of I into a corresponding pre-condition.

We then define semantic type-spec judgements:

$$\mathsf{E}; \mathsf{L} \mid \mathsf{T}_1 \models \mathit{I} \dashv\models x.\, \mathsf{T}_2 \rightsquigarrow \Phi$$

and then prove the fundamental theorem and adequacy.

Note: adequacy is now more interesting, because it also states that logical assertions are not violated.

Example rules

$$\mathsf{E}; \mathsf{L} \mid p_1 \lhd \textbf{int}, p_2 \lhd \textbf{int} \vdash p_1 \leq p_2 \dashv x.\, x \lhd \textbf{bool} \rightsquigarrow \lambda\ P\ (n_1, n_2).\ P(n_1 \leq n_2)$$

I.e., we translate `x1 <= x2` into $p_1 \leq p_2$ in the functional language.

Example rules

$$\mathsf{E};\mathsf{L} \mid p_1 \lhd \mathbf{int}, p_2 \lhd \mathbf{int} \vdash p_1 \leq p_2 \dashv x.\, x \lhd \mathbf{bool} \rightsquigarrow \lambda\ P\ (n_1, n_2).\ P(n_1 \leq n_2)$$

I.e., we translate `x1 <= x2` into $p_1 \leq p_2$ in the functional language.

$$\mathsf{E};\mathsf{L} \vdash p \lhd \mathbf{own}_n\, \tau \overset{\mathbf{ctx}}{\Rightarrow} p \lhd \&^{\kappa}_{\mathbf{mut}}\, \tau, p \lhd^{\dagger\kappa} \mathbf{own}_n\, \tau \rightsquigarrow \lambda\ P\ x.\ \forall \alpha.\ P((x, \alpha), \alpha)$$

I.e., if `x: Box<T>`, we translate `let b = &mut *x` into
`let alph = any in let b = (x, alph) in let x = alph in`.

Verifying Rust code by translation to functional code
Jacques-Henri Jourdan
A functional translation
Rust as a functional language
Mutable borrows
Causality loops
Proving soundness of the translation

Definition of the semantic judgement

Recall:

$$\mathsf{E}; \mathsf{L} \mid \mathsf{T}_1 \models I \Rightarrow x.\, \mathsf{T}_2 :=$$
$$\forall t.\, \{[\![\mathsf{E}]\!] * [\![\mathsf{L}]\!] * [\mathrm{Na} : t] * [\![\mathsf{T}_1]\!](t)\} \;\; I \;\; \{v.\, [\![\mathsf{L}]\!] * [\mathrm{Na} : t] * [\![\mathsf{T}_2]\!]_{[x \leftarrow v]}(t)\}$$

Definition of the semantic judgement

Recall:

$$\mathsf{E}; \mathsf{L} \mid \mathsf{T}_1 \models I \Rightarrow x.\, \mathsf{T}_2 :=$$
$$\forall t.\, \{[\![\mathsf{E}]\!] * [\![\mathsf{L}]\!] * [\mathrm{Na} : t] * [\![\mathsf{T}_1]\!](t)\} \;\; I \;\; \{v.\, [\![\mathsf{L}]\!] * [\mathrm{Na} : t] * [\![\mathsf{T}_2]\!]_{[x \leftarrow v]}(t)\}$$

In the new judgements, we specify that the predicate transformer is valid, too:

$$\mathsf{E}; \mathsf{L} \mid \mathsf{T}_1 \models I \Rightarrow x.\, \mathsf{T}_2 \rightsquigarrow \Phi \approxeq$$
$$\forall t P.\, \{\exists \bar{v}_1.\, \Phi(P)(\bar{v}_1) * \cdots * [\![\mathsf{T}_1]\!](t, \bar{v}_1)\} \;\; I \;\; \{v.\, \exists \bar{v}_2.\, P(\bar{v}_2) * \cdots * [\![\mathsf{T}_2]\!]_{[x \leftarrow v]}(t)(\bar{v}_2)\}$$

Notes:

- $\bar{v}_1$ and $\bar{v}_2$ are logical values, used in the specifications (not program values).
- Type and context interpretation depend both on program values and logical values.

The problem with prophecies

How do we prove:

$$\mathsf{E}; \mathsf{L} \vdash p \lhd \mathbf{own}_n\, \tau \overset{\mathsf{ctx}}{\Rrightarrow} p \lhd \&^{\kappa}_{\mathbf{mut}}\, \tau, p \lhd^{\dagger\kappa} \mathbf{own}_n\, \tau \rightsquigarrow \lambda\ P\ x.\ \forall \alpha.\ P((x, \alpha), \alpha)$$

The $\forall$ quantifier appears as a precondition.
We need to instantiate it with the final value of the borrow, but we don't know it yet!

Modeling prophecies

Solution: embed Φ in a reader monad that quantifies over all possible futures values of prophecies π, and values may depend on them:

$$\mathsf{E}; \mathsf{L} \mid \mathsf{T}_1 \models I \Rrightarrow x.\, \mathsf{T}_2 \rightsquigarrow \Phi \cong$$
$$\forall tP.\, \{\exists \bar{v}_1.\, \langle \pi.\ \Phi(P)(\bar{v}_1(\pi))\rangle * \cdots\}\ \ I\ \ \{v.\, \exists \bar{v}_2.\, \langle \pi.\ P(\bar{v}_2(\pi))\rangle * \cdots\}$$

Then, we can instantiate the quantifier with a (prophetic) value that depends on π. We also prove sound rules to reason with $\langle \pi.\ \ldots\rangle$ that make sure there is no causality circle.

Conclusion

We presented a technique for proving correct programs that manipulate pointers, without separation logic, and without stating complex non-aliasing assertions.

Instead, we exploit the powerful properties of Rust's type system.

Handling mutable borrows require using a notion of prophecies, whose soundness is subtle.
We sketched a soundness proof, generalizing the soundness proof of the type system.

Conclusion

We presented a technique for proving correct programs that manipulate pointers, without separation logic, and without stating complex non-aliasing assertions.

Instead, we exploit the powerful properties of Rust's type system.

Handling mutable borrows require using a notion of prophecies, whose soundness is subtle.
We sketched a soundness proof, generalizing the soundness proof of the type system.

Follow-up: how to handle interior mutability and raw pointers?
Answer: this requires a new notion: ghost ownership in Rust programs.

Agenda

- Next week 2026/03/11: **exam**
- Yesterday 2026/03/03: deadline for programming project